

Biblioteca digital

por: Nicolás Krebs Mella

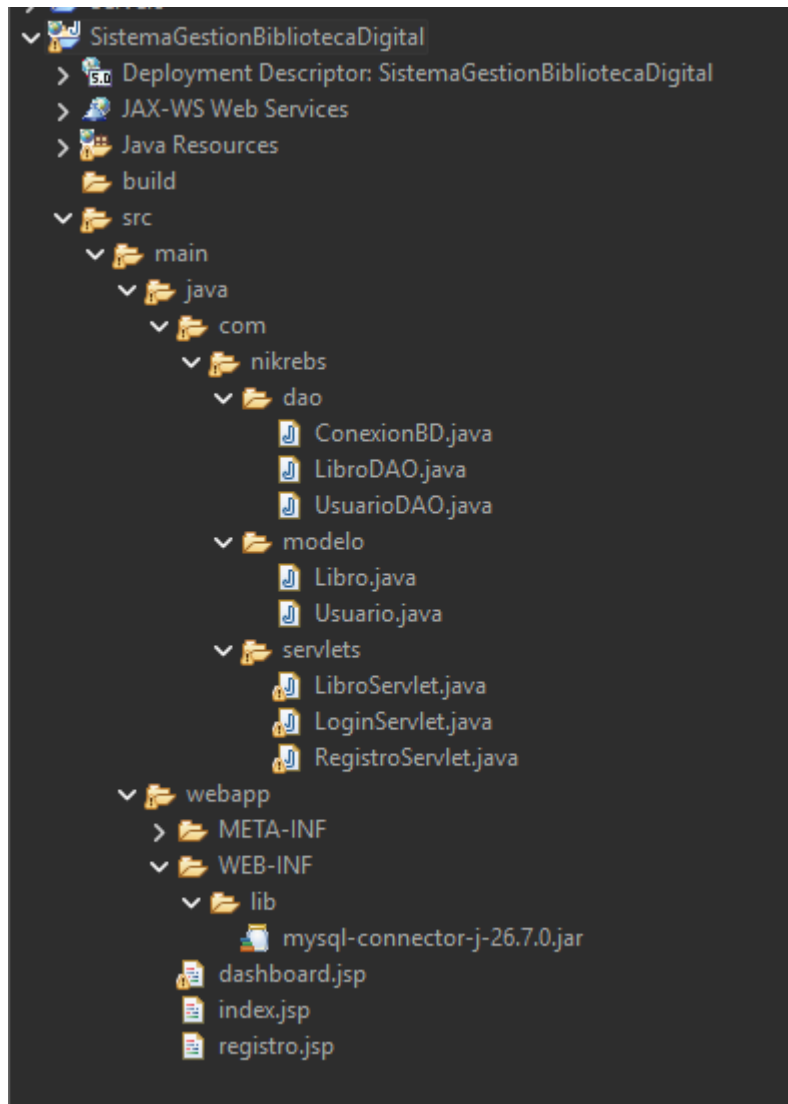
Busca ser una página web dinámica para poder administrar una biblioteca digital, pudiendo los usuarios poder pedir prestado los libros y/o devolverlos, mientras que el administrador puede ver quién tiene el libro, agregar libros al catálogo y eliminarlos del mismo; catálogo conectado a una base de datos SQL (para este ejemplo, se crea una base temporal en MySQL Workbench para hacer pruebas, pero debería poder ser escalable siguiendo el diseño inicial)

Índice

Índice.....	2
1. Jerarquía de archivos:.....	3
2. Clases:.....	4
2.1 Usuario.....	4
2.2 UsuarioDAO.....	5
2.3 Libro.....	6
2.4 LibroDAO.....	7
3. Servlets:.....	10
3.1 LoginServlet.....	10
3.2 RegistroServlet.....	10
3.3 LibroServlet.....	11
4. Capa visible.....	12
4.1 index.jsp.....	12
4.2 Registro.jsp.....	14
4.3 Dashboard.jsp.....	17
5. Anexo.....	21
5.1 Tomcat 10.1.....	21
5.2 Base de datos.....	22

1. Jerarquía de archivos:

orden de los archivos, com.nikrebs.ejemplo es la idea general



2. Clases:

Definimos las clases usuario, usuarioDAO, libro, libroDAO para su implementación correspondiente. Dado la simpleza de la idea, los administradores podrían ser una clase heredada de admin, y de esta forma jerarquizar de una mejor forma si se quieren añadir futuras variantes de usuario o administradores. Esta idea fue descartada para mantener la simpleza del diseño, teniendo en cuenta solo de la dualidad entre user/admin, no hay más variantes.

2.1 Usuario

```
1 package com.nikrebs.modelo;
2 import java.util.ArrayList;
3
4 public class Usuario {
5     private int id;
6     private String nombre;
7     private String email;
8     private String password;
9     private boolean esAdmin;
10    private ArrayList<Libro> librosPrestados = new ArrayList<>();
11
12    public Usuario(int id, String nombre, String email, String password, boolean esAdmin) {
13        this.id = id;
14        this.nombre = nombre;
15        this.email = email;
16        this.password = password;
17        this.esAdmin = esAdmin;
18    }
19
```

y sus respectivos getters and setters:

```
20 // Getters y Setters
21 public int getId() {
22     return id;
23 }
24 public String getNombre() {
25     return nombre;
26 }
27 public String getEmail() {
28     return email;
29 }
30 public String getPassword() {
31     return password;
32 }
33 public boolean isEsAdmin() {
34     return esAdmin;
35 }
36 public ArrayList<Libro> getLibrosPrestados() {
37     return librosPrestados;
38 }
39 public void addLibroPrestado(Libro libro) {
40     this.librosPrestados.add(libro);
41 }
42
43 }
```

2.2 UsuarioDAO

```
1 package com.nikrebs.dao;
2 import com.nikrebs.modelo.Usuario;
3
4
5 public class UsuarioDAO {
6     private String url = "jdbc:mysql://localhost:3306/biblioteca_db"; // base de datos de prueba
7     private String user = "root"; // user
8     private String pass = "1234"; // pw
9
10    public Usuario autenticar(String email, String password) {
11        try {
12            Class.forName("com.mysql.cj.jdbc.Driver");
13            Connection con = DriverManager.getConnection(url, user, pass);
14            PreparedStatement ps = con.prepareStatement("SELECT * FROM usuarios WHERE email=? AND password=?");
15            ps.setString(1, email);
16            ps.setString(2, password);
17            ResultSet rs = ps.executeQuery();
18
19            if (rs.next()) {
20                return new Usuario(rs.getInt("id"), rs.getString("nombre"), rs.getString("email"), rs.getString("password"), rs.getBoolean("es_admin"));
21            }
22            con.close();
23        } catch (Exception e) {
24            e.printStackTrace();
25        }
26        return null;
27    }
28 }
```

y su función para registrar nuevos usuarios.

```
28 public boolean registrarUsuario(Usuario u) {
29     String sql = "INSERT INTO usuarios (nombre, email, password, es_admin) VALUES (?, ?, ?, false)";
30     try (Connection con = ConexionBD.getConexion();
31         PreparedStatement ps = con.prepareStatement(sql)) {
32
33         ps.setString(1, u.getNombre());
34         ps.setString(2, u.getEmail());
35         ps.setString(3, u.getPassword());
36
37         int filasInsertadas = ps.executeUpdate();
38         return filasInsertadas > 0; // true si guardado
39     } catch (Exception e) { // guardado
40         e.printStackTrace();
41         return false;
42     }
43 }
44
45
46
47 }
```

2.3 Libro

```
1 package com.nikrebs.modelo;
2
3 public class Libro {
4     private int id; // autoincrement en sql, no su numero en lista
5     private String titulo;
6     private String autor;
7     private boolean disponible;
8     private int idUsuario; //tracker de quien tiene el libro
9     private String nombreUsuarioPrestamo;
10
11     public Libro(int id, String titulo, String autor, boolean disponible) {
12         this.id = id;
13         this.titulo = titulo;
14         this.autor = autor;
15         this.disponible = disponible;
16     }
17 }
```

Es el libro quien lleva el nombre de quién lo tiene prestado, importante para la lógica de muestra del dashboard más adelante.

y sus respectivos getters and setters:

```
18 // Getters y Setters zzz
19 public int getId() {
20     return id;
21 }
22 public void setId(int id) {
23     this.id = id;
24 }
25 public String getTitulo() {
26     return titulo;
27 }
28 public void setTitulo(String titulo) {
29     this.titulo = titulo;
30 }
31 public String getAutor() {
32     return autor;
33 }
34 public void setAutor(String autor) {
35     this.autor = autor;
36 }
37 public boolean isDisponible() {
38     return disponible;
39 }
40 public void setDisponible(boolean disponible) {
41     this.disponible = disponible;
42 }
43 public int getIdUsuario() {
44     return idUsuario;
45 }
46 public void setIdUsuario(int idUsuario) {
47     this.idUsuario = idUsuario;
48 }
49 public String getNombreUsuarioPrestamo() {
50     return nombreUsuarioPrestamo;
51 }
52 public void setNombreUsuarioPrestamo(String nombreUsuarioPrestamo) {
53     this.nombreUsuarioPrestamo = nombreUsuarioPrestamo;
54 }
55 }
56 }
```

2.4 LibroDAO

```
1 package com.nikrebs.dao;
2 import com.nikrebs.modelo.Libro;
3
4
5
6
7 public class LibroDAO {
8     private String url = "jdbc:mysql://localhost:3306/biblioteca_db";
9     private String user = "root";
10    private String pass = "1234";
11
12    public List<Libro> listarTodos() {
13        List<Libro> lista = new ArrayList<>();
14        //CONSULTA SQL CON LEFT JOIN
15        String sql = "SELECT l.*, u.nombre AS nombre_usuario FROM libros l LEFT JOIN usuarios u ON l.id_usuario = u.id";
16
17        try (Connection con = ConexionBD.getConnection();
18            PreparedStatement ps = con.prepareStatement(sql);
19            ResultSet rs = ps.executeQuery()) {
20
21            while (rs.next()) {
22                Libro libro = new Libro(
23                    rs.getInt("id"),
24                    rs.getString("titulo"),
25                    rs.getString("autor"),
26                    rs.getBoolean("disponible")
27                );
28                libro.setIdUsuario(rs.getInt("id_usuario"));
29
30                //Guardamos el nombre del usuario (si es nulo, guardará null)
31                libro.setNombreUsuarioPrestamo(rs.getString("nombre_usuario"));
32
33                lista.add(libro);
34            }
35        } catch (Exception e) { e.printStackTrace(); }
36        return lista;
37    }
38 }
```

y sus respectivas funciones

2.4.1 Agregar libro

```
39 public void agregarLibro(Libro l) {
40     try {
41         Connection con = DriverManager.getConnection(url, user, pass);
42         PreparedStatement ps = con.prepareStatement("INSERT INTO libros (titulo, autor, disponible) VALUES (?, ?, ?)");
43         ps.setString(1, l.getTitulo());
44         ps.setString(2, l.getAutor());
45         ps.setBoolean(3, true);
46         ps.executeUpdate();
47         con.close();
48     } catch (Exception e) { e.printStackTrace(); }
49 }
50 }
```

Si bien es una función exclusiva administrativa, debe existir para poder usarse más adelante

2.4.2 prestarLibro

```
51 public void prestarLibro(int idLibro, int idUsuario) {
52     try {
53         Connection con = DriverManager.getConnection(url, user, pass);
54         PreparedStatement ps = con.prepareStatement("UPDATE libros SET disponible = FALSE, id_usuario = ? WHERE id = ?");
55         ps.setInt(1, idUsuario);
56         ps.setInt(2, idLibro);
57         ps.executeUpdate();
58         con.close();
59     } catch (Exception e) { e.printStackTrace(); }
60 }
```

2.4.3 devolverLibro

```
61 public void devolverLibro(int idLibro, int idUsuario) {
62     try {
63         Connection con = DriverManager.getConnection(url, user, pass);
64         //update libro devuelto
65         PreparedStatement ps = con.prepareStatement("UPDATE libros SET disponible = TRUE, id_usuario = NULL WHERE id = ? AND id_usuario = ?");
66         ps.setInt(1, idLibro);
67         ps.setInt(2, idUsuario);
68         ps.executeUpdate();
69         con.close();
70     } catch (Exception e) { e.printStackTrace(); }
71 }
```

2.4.4 eliminarLibro

```
72●    public void eliminarLibro(int idLibro) {
73        String sql = "DELETE FROM libros WHERE id = ?";
74        try (Connection con = ConexionBD.getConnection();
75            PreparedStatement ps = con.prepareStatement(sql)) {
76            ps.setInt(1, idLibro);
77            ps.executeUpdate();
78
79        } catch (Exception e) {
80            e.printStackTrace();
81        }
82    }
83 }
```

2.5 ConexiónBD

puente entre la base de datos y la página web dinámica.

```
1 package com.nikrebs.dao;
2
3● import java.sql.Connection;
4
5
6
7● /**
8  * Clase que gestiona la conexión a la base de datos usando el Patrón Singleton.
9  */
10 public class ConexionBD {
11
12     // biblioteca_db es mi base de datos
13     private static final String URL = "jdbc:mysql://localhost:3306/biblioteca_db";
14     private static final String USUARIO = "root";
15     private static final String PASSWORD = "1234";
16     private static Connection conexion = null; //validar conexion
17
18● /**
19  * Conexión a SQL
20  */
21● private ConexionBD() {
22●     try {
23
24         Class.forName("com.mysql.cj.jdbc.Driver");
25
26         // Output en consola para saber que está todo correcto en la bd
27         conexion = DriverManager.getConnection(URL, USUARIO, PASSWORD);
28         System.out.println("¡Conexión a MySQL establecida con éxito!");
29
30●     } catch (ClassNotFoundException e) {
31         System.out.println("Error crítico: No se encontró el Driver JDBC de MySQL.");
32         e.printStackTrace();
33●     } catch (SQLException e) {
34         // apareció más veces de lo que me esperaba, resulta que tenía mal la pw en los dao xd
35
36         System.out.println("Error crítico: Fallo al intentar conectar con la base de datos.");
37         e.printStackTrace();
38     }
39 }
40 }
```

getConexion

```
40
41●  /**
42   * @return La conexión activa a la base de datos.
43   */
44●  public static Connection getConexion() {
45●      try {
46●          if (conexion == null || conexion.isClosed()) {
47              new ConexionBD();
48          }
49●      } catch (SQLException e) {
50          System.out.println("Error al verificar el estado de la conexión.");
51          e.printStackTrace();
52      }
53      return conexion;
54  }
55 }
```

3. Servlets:

Principales puentes de conexión entre la base de datos, y el cómo se quiere dar uso con la misma. Debemos conectar lo recibido en la web de formularios para el registro/uso de usuario, con la base de datos correspondiente, asimismo con la presentación de los libros más adelante.

3.1 LoginServlet

```
1 package com.nikrebs.servlets;
2 import jakarta.servlet.ServletException;
3
4
5
6
7
8
9 @WebServlet("/LoginServlet")
10 public class LoginServlet extends HttpServlet {
11     protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
12         String email = request.getParameter("email");
13         String pass = request.getParameter("password");
14
15         UsuarioDAO dao = new UsuarioDAO();
16         Usuario user = dao.autenticar(email, pass);
17
18         if (user != null) {
19             HttpSession session = request.getSession();
20             session.setAttribute("usuario", user);
21             response.sendRedirect("LibroServlet"); // Redirige al dashboard de libros
22         } else {
23             response.sendRedirect("index.jsp?error=1");
24         }
25     }
26 }
27
```

3.2 RegistroServlet

```
1 package com.nikrebs.servlets;
2
3 import jakarta.servlet.ServletException;
4
5
6
7
8
9
10 @WebServlet("/RegistroServlet")
11 public class RegistroServlet extends HttpServlet {
12
13     protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
14         // Obtenemos los datos del formulario
15         String nombre = request.getParameter("nombre");
16         String email = request.getParameter("email");
17         String password = request.getParameter("password");
18
19         // usuario temporal, reemplazamos luego con la info de registro.jsp
20         Usuario nuevoUsuario = new Usuario(0, nombre, email, password, false);
21         UsuarioDAO dao = new UsuarioDAO();
22
23         // Intentamos registrar
24         if (dao.registrarUsuario(nuevoUsuario)) {
25             // Si funciona, lo mandamos al login con un mensaje de éxito
26             response.sendRedirect("index.jsp?registro=exito");
27         } else {
28             // Si falla, lo devolvemos al registro con error
29             response.sendRedirect("registro.jsp?error=1");
30         }
31     }
32 }
```

3.3 LibroServlet

un poco más extenso de lo habitual, pues es la principal funcionalidad de esta pagina web dinámica. Si el usuario de alguna manera puede llenar e intentar mandar el formulario para poder agregar el libro, de todas formas aquí deberíamos de chequear que la información haya sido enviada por un administrador.

```
1 package com.nikrebs.servlets;
2 import jakarta.servlet.ServletException;
10
11 @WebServlet("/LibroServlet")
12 public class LibroServlet extends HttpServlet {
13     private LibroDAO dao = new LibroDAO();
14     // protected
15     protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
16         List<Libro> libros = dao.listarTodos();
17         request.setAttribute("libros", libros);
18         request.getRequestDispatcher("dashboard.jsp").forward(request, response);
19     }
20
21     protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
22         String accion = request.getParameter("accion");
23         HttpSession session = request.getSession();
24         Usuario user = (Usuario) session.getAttribute("usuario");
25
26         if (user != null) { //osea que si hay usuario
27             if ("agregar".equals(accion) && user.isEsAdmin()) { //doblecheck de que sea admin, el primero es en dashboard
28                 String titulo = request.getParameter("titulo");
29                 String autor = request.getParameter("autor");
30                 dao.agregarLibro(new Libro(0, titulo, autor, true));
31             } else if ("prestar".equals(accion)) {
32                 int idLibro = Integer.parseInt(request.getParameter("idLibro"));
33                 dao.prestarLibro(idLibro, user.getId());
34             }
35             else if ("devolver".equals(accion)) {
36                 int idLibro = Integer.parseInt(request.getParameter("idLibro"));
37                 dao.devolverLibro(idLibro, user.getId());
38             }
39             else if ("eliminar".equals(accion) && user.isEsAdmin()) {
40                 int idLibro = Integer.parseInt(request.getParameter("idLibro"));
41                 dao.eliminarLibro(idLibro);
42             }
43         }
44
45         response.sendRedirect("LibroServlet");
46     }
47 }
48 }
```

4. Capa visible

4.1 index.jsp

html con bootstrap para poder dar un diseño amigable y responsivo. En modo oscuro, para cuidar la vista.

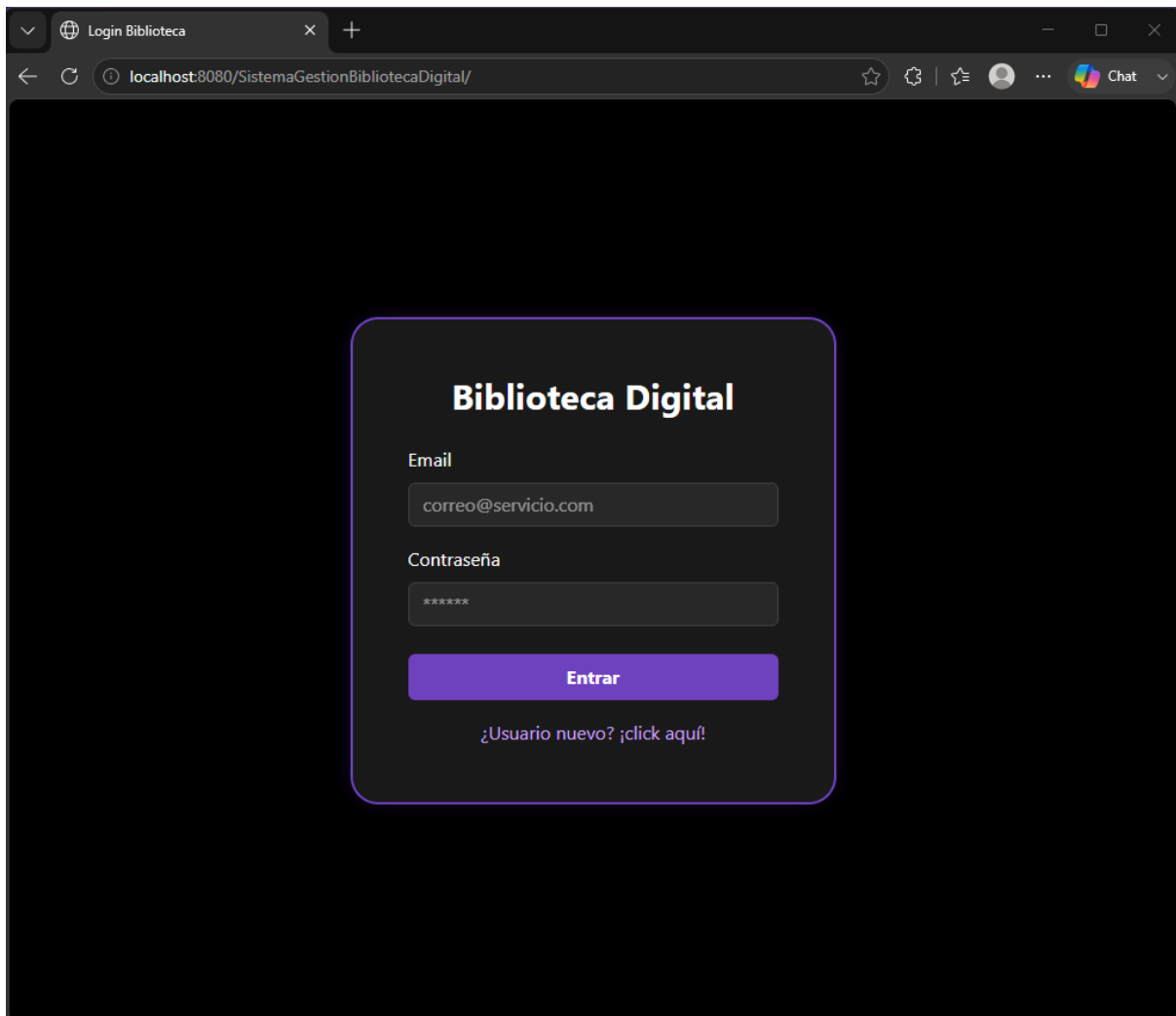
```
1 <%@ page contentType="text/html; charset=UTF-8" language="java" %>
2 <!DOCTYPE html>
3 <html lang="es">
4 <head>
5   <meta charset="UTF-8">
6   <title>Login Biblioteca</title>
7   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet">
8   <style>
9     body {
10       background-color: #000000;
11       color: #ffffff;
12     }
13     .login-container {
14       background-color: #1a1a1a; /* gris oscuro */
15       border: 2px solid #6f42c1; /* Borde morado */
16       border-radius: 1.5rem; /* Bordes redondeados */
17       box-shadow: 0 0 20px rgba(111, 66, 193, 0.2); /* brillo bajo */
18     }
19     .btn-purple {
20       background-color: #6f42c1;
21       color: #ffffff;
22       border: none;
23       transition: 0.3s;
24     }
25     .btn-purple:hover {
26       background-color: #59339d;
27       color: #ffffff;
28     }
29     .form-control {
30       background-color: #2a2a2a;
31       color: #ffffff;
32       border: 1px solid #444;
33     }
34     .form-control::placeholder {
35       color: #ffffff;
36       opacity: 0.5; /* mejor opacidad */
37     }
38     .form-control:focus {
39       background-color: #2a2a2a;
40       color: #ffffff;
41       border-color: #6f42c1;
42       box-shadow: 0 0 0.25rem rgba(111, 66, 193, 0.25);
43     }
44   </style>
```

```

46● <body class="d-flex align-items-center justify-content-center vh-100">
47● <div class="Login-container p-5 w-100" style="max-width: 420px;">
48   <h2 class="text-center mb-4 text-white fw-bold">Biblioteca Digital</h2>
49
50   <% if(request.getParameter("error") != null) { %>
51●   <div class="alert alert-danger p-2 text-center" role="alert">
52     Ingrese usuario y contraseña reales
53   </div>
54   <% } %>
55
56   <form action="LoginServlet" method="POST">
57●   <div class="mb-3">
58     <label for="email" class="form-label">Email</label>
59     <input type="email" class="form-control" id="email" name="email" placeholder="correo@servicio.com" required>
60   </div>
61●   <div class="mb-4">
62     <label for="password" class="form-label">Contraseña</label>
63     <input type="password" class="form-control" id="password" name="password" placeholder="*****" required>
64   </div>
65   <button type="submit" class="btn btn-purple w-100 py-2 fw-bold">Entrar</button>
66 </form>
67 <!-- Añade esto justo debajo de </form> en tu index.jsp -->
68 <% if("exito".equals(request.getParameter("registro"))) { %>
69●   <div class="alert alert-success p-2 mt-3 text-center" role="alert">
70     ¡Éxito! ahora puedes iniciar sesión.
71   </div>
72   <% } %>
73
74●   <div class="text-center mt-3">
75     <a href="registro.jsp" class="text-decoration-none" style="color: #cda4ff;">¿Usuario nuevo? ¡click aquí!</a>
76   </div>
77 </div>
78 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"></script>
79 </body>
80 </html>

```

Nos preocupamos de que el usuario pueda ingresar y conectar correctamente los datos.



4.2 Registro.jsp

```
1 <%@ page contentType="text/html; charset=UTF-8" language="java" %>
2 <!DOCTYPE html>
3 <html lang="es">
4 <head>
5   <meta charset="UTF-8">
6   <title>Registro - Biblioteca Digital</title>
7   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
8   <style>
9     body {
10       background-color: #000000;
11       color: #ffffff;
12     }
13     .login-container {
14       background-color: #1a1a1a;
15       border: 2px solid #6f42c1;
16       border-radius: 1.5rem;
17       box-shadow: 0 0 20px rgba(111, 66, 193, 0.2);
18     }
19     .btn-purple {
20       background-color: #6f42c1;
21       color: #ffffff;
22       border: none;
23       transition: 0.3s;
24     }
25     .btn-purple:hover {
26       background-color: #59339d;
27       color: #ffffff;
28     }
29     .form-control {
30       background-color: #2a2a2a;
31       color: #ffffff;
32       border: 1px solid #444;
33     }
34     .form-control:focus {
35       background-color: #2a2a2a;
36       color: #ffffff;
37       border-color: #6f42c1;
38       box-shadow: 0 0 0.25rem rgba(111, 66, 193, 0.25);
39     }
40     .form-control::placeholder {
41       color: #a991d4;
42       opacity: 0.5;
43     }
44   </style>
45 </head>
```

Seguimos el mismo diseño anterior.

```
46 <body class="d-flex align-items-center justify-content-center vh-100">
47   <div class="login-container p-5 w-100" style="max-width: 420px;">
48     <h2 class="text-center mb-4 text-white fw-bold">Crear Cuenta</h2>
49
50     <% if(request.getParameter("error") != null) { %>
51       <div class="alert alert-danger p-2 text-center" role="alert">
52         Error: El correo ya existe o los datos son inválidos.
53       </div>
54     <% } %>
55
56     <form action="RegistroServlet" method="POST">
57       <div class="mb-3">
58         <label for="nombre" class="form-label">Nombre Completo</label>
59         <input type="text" class="form-control" id="nombre" name="nombre" placeholder="Ej. Juan Pérez" required>
60       </div>
61       <div class="mb-3">
62         <label for="email" class="form-label">Email</label>
63         <input type="email" class="form-control" id="email" name="email" placeholder="correo@ejemplo.com" required>
64       </div>
65       <div class="mb-4">
66         <label for="password" class="form-label">Contraseña</label>
67         <input type="password" class="form-control" id="password" name="password" placeholder="Tu contraseña secreta" required>
68       </div>
69       <button type="submit" class="btn btn-purple w-100 py-2 fw-bold">Registrarse</button>
70     </form>
71
72     <div class="text-center mt-3">
73       <a href="index.jsp" class="text-decoration-none" style="color: #cda4ff;">¿Ya tienes cuenta? Inicia sesión aquí</a>
74     </div>
75   </div>
76 </body>
77 </html>
```

Nos preocupamos del formulario correspondiente y una manera de volver a index.

The image shows a web browser window with the address bar displaying 'localhost:8080/SistemaGestionBibliotecaDigital/registro.jsp'. The page has a dark background and features a central registration form with a purple border. The form is titled 'Crear Cuenta' in white bold text. It contains three input fields: 'Nombre Completo' with the placeholder 'Ej. Juan Pérez', 'Email' with the placeholder 'correo@ejemplo.com', and 'Contraseña' with the placeholder 'Tu contraseña secreta'. Below these fields is a purple button labeled 'Registrarse'. At the bottom of the form, there is a link that says '¿Ya tienes cuenta? Inicia sesión aquí'.

Registro - Biblioteca Digital

localhost:8080/SistemaGestionBibliotecaDigital/registro.jsp

Crear Cuenta

Nombre Completo

Ej. Juan Pérez

Email

correo@ejemplo.com

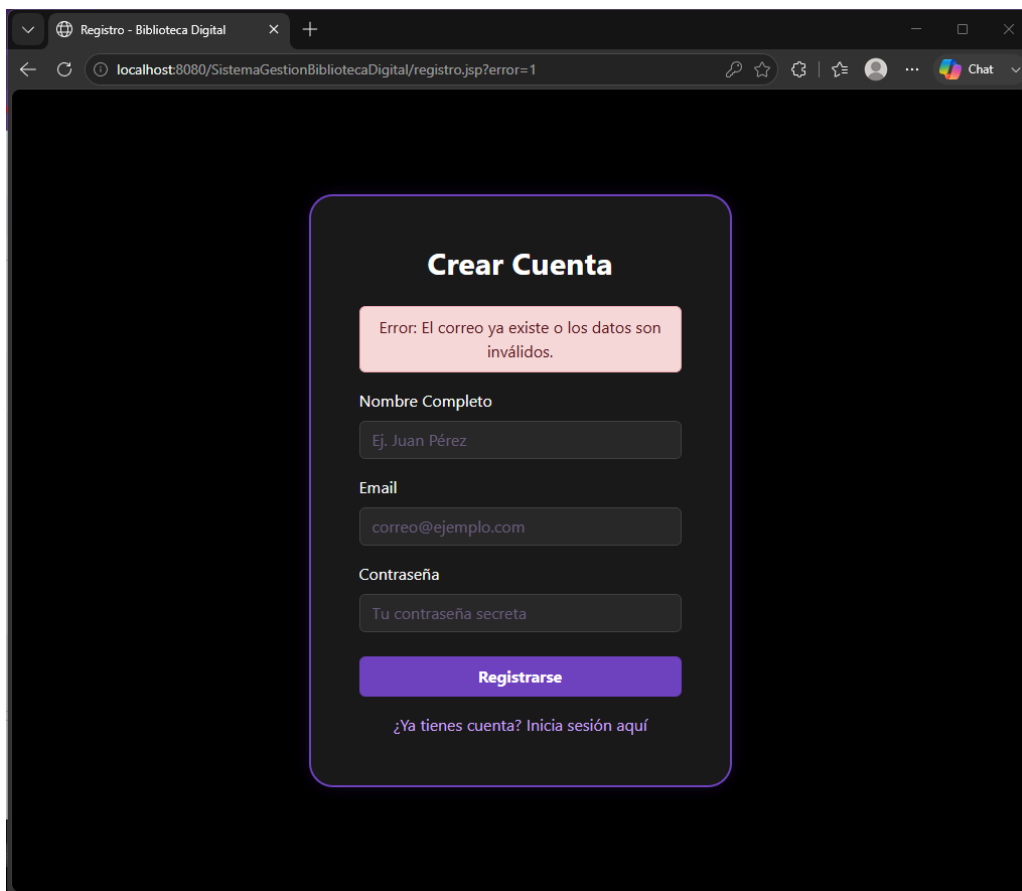
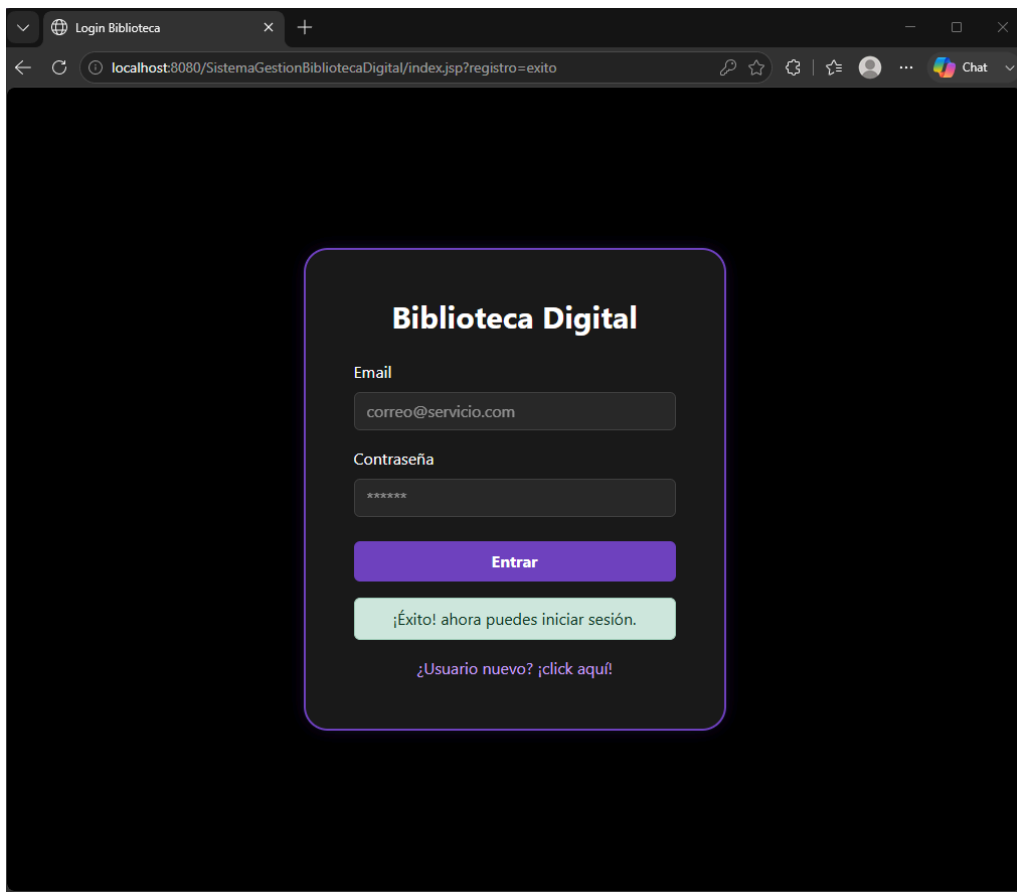
Contraseña

Tu contraseña secreta

Registrarse

[¿Ya tienes cuenta? Inicia sesión aquí](#)

Registrarse correctamente lleva devuelta a index.jsp con un mensaje nuevo



intentar usar el mismo correo una segunda vez, nos limpia los campos y entre un error.

4.3 Dashboard.jsp

```
1 <%@ page contentType="text/html; charset=UTF-8" language="java" %>
2 <%@ page import="java.util.List" %>
3 <%@ page import="com.nikrebs.modelo.Libro" %>
4 <%@ page import="com.nikrebs.modelo.Usuario" %>
5 <%
6     Usuario user = (Usuario) session.getAttribute("usuario");
7     if(user == null) { response.sendRedirect("index.jsp"); return; }
8     List<Libro> libros = (List<Libro>) request.getAttribute("libros");
9 %>
10 <!DOCTYPE html>
11 <html lang="es">
12 <head>
13     <meta charset="UTF-8">
14     <title>¡Biblioteca!</title>
15     <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
16     <style>
17         body {
18             background-color: #000000;
19             color: #ffffff;
20         }
21         .admin-panel {
22             background-color: #1a1a1a;
23             border-left: 4px solid #6f42c1;
24         }
25         /* Modo oscuro */
26         .table-dark {
27             --bs-table-bg: #121212;
28             --bs-table-striped-bg: #1a1a1a;
29             border-color: #333;
30         }
31         .table-custom-header {
32             border-bottom: 2px solid #6f42c1 !important;
33         }
34         .form-control::placeholder {
35             color: #ffffff;
36             opacity: 0.5;
37         }
38     </style>
39 </head>
```

la inicio se valida de que existe efectivamente un usuario activo.

```
43     .btn-purple:hover {
44         background-color: #59339d;
45         color: #ffffff;
46     }
47     .form-control {
48         background-color: #2a2a2a;
49         color: #ffffff;
50         border: 1px solid #444;
51     }
52     .form-control:focus {
53         background-color: #2a2a2a;
54         color: #ffffff;
55         border-color: #6f42c1;
56         box-shadow: 0 0 0 0.25rem rgba(111, 66, 193, 0.25);
57     }
58 </style>
59 </head>
```

aquí se requiere una mayor customización para el uso del dashboard más adelante.

```

60<body>
61<div class="container mt-5">
62
63    <!-- header -->
64<div class="d-flex justify-content-between align-items-center mb-5 border-bottom border-secondary pb-3">
65    <h2>Hola, <span style="color: #cda4ff;"><%= user.getNombre() %></span>
66    <span class="fs-5 text-muted"><%= user.isEsAdmin() ? "Administrador/a" : "" %></span>
67    </h2>
68    <!-- cierre sesion boton -->
69    <a href="index.jsp" class="btn btn-outline-light btn-sm">Cerrar Sesión</a>
70</div>
71
72    <!-- Botones nuevos si el usuario es admin -->
73    <% if(user.isEsAdmin()) { %>
74    <div class="admin-panel p-4 rounded-4 mb-5 shadow-sm">
75        <h4 class="mb-3">Añadir Nuevo Libro</h4>
76        <form action="LibroServlet" method="POST" class="row g-3 align-items-center">
77            <input type="hidden" name="accion" value="agregar">
78            <div class="col-md-5">
79                <input type="text" class="form-control" name="titulo" placeholder="Título del libro" required>
80            </div>
81            <div class="col-md-5">
82                <input type="text" class="form-control" name="autor" placeholder="Autor del libro" required>
83            </div>
84            <div class="col-md-2">
85                <button type="submit" class="btn btn-purple w-100 fw-bold">Añadir Libro</button>
86            </div>
87        </form>
88    </div>
89    <% } %>
90

```

Si el usuario es el administrador, tiene un contenedor adicional para poder agregar libros.

Hola, Administrador (Administrador/a) Cerrar Sesión

Añadir Nuevo Libro

Título del libro Autor del libro Añadir Libro

Catálogo de Libros

ID	TÍTULO	AUTOR	ESTADO	ACCIÓN
2	1984	George Orwell	Disponible	Pedir Prestado
4	Nacidos de la bruma	Brandon Sanderson	Prestado	Prestado a: Juan Perez
5	El Despertar de Rumi	Cristian Sarmiento	Prestado	Prestado a: Nico Krebs
6	Breve resumen de la historia de Chile	Ricardo Krebs	Disponible	Pedir Prestado

Asimismo, el administrador puede ver quién tiene prestado el libro actualmente y puede eliminar libros. El ID de los libros, es incremental y no corresponde a una lista enumerada pero si a su ID en el “stock”.

Parte superior del formulario y como se desarrolla verticalmente

```

91      <!-- formulario lista libros -->
92      <h3 class="mb-4">Catálogo de Libros</h3>
93      <div class="table-responsive rounded-4 shadow-lg overflow-hidden">
94          <table class="table table-dark table-striped table-hover align-middle mb-0">
95              <thead class="table-custom-header text-uppercase text-secondary small">
96                  <tr>
97                      <th class="ps-4">ID</th>
98                      <th>Título</th>
99                      <th>Autor</th>
100                     <th>Estado</th>
101                     <th class="text-center pe-4">Acción</th>
102                 </tr>
103             </thead>
104             <tbody>
105                 <% for(Libro l : libros) { %>
106                 <tr>
107                     <td class="ps-4 text-white"><%= l.getId() %></td>
108                     <td class="fw-bold"><%= l.getTitulo() %></td>
109                     <td><%= l.getAutor() %></td>
110                     <td>
111                         <% if(l.isDisponible()) { %>
112                             <span class="badge bg-success">Disponible</span>
113                         <% } else { %>
114                             <span class="badge bg-secondary">Prestado</span>
115                         <% } %>
116                     </td>
117                     <td class="text-center pe-4">

```

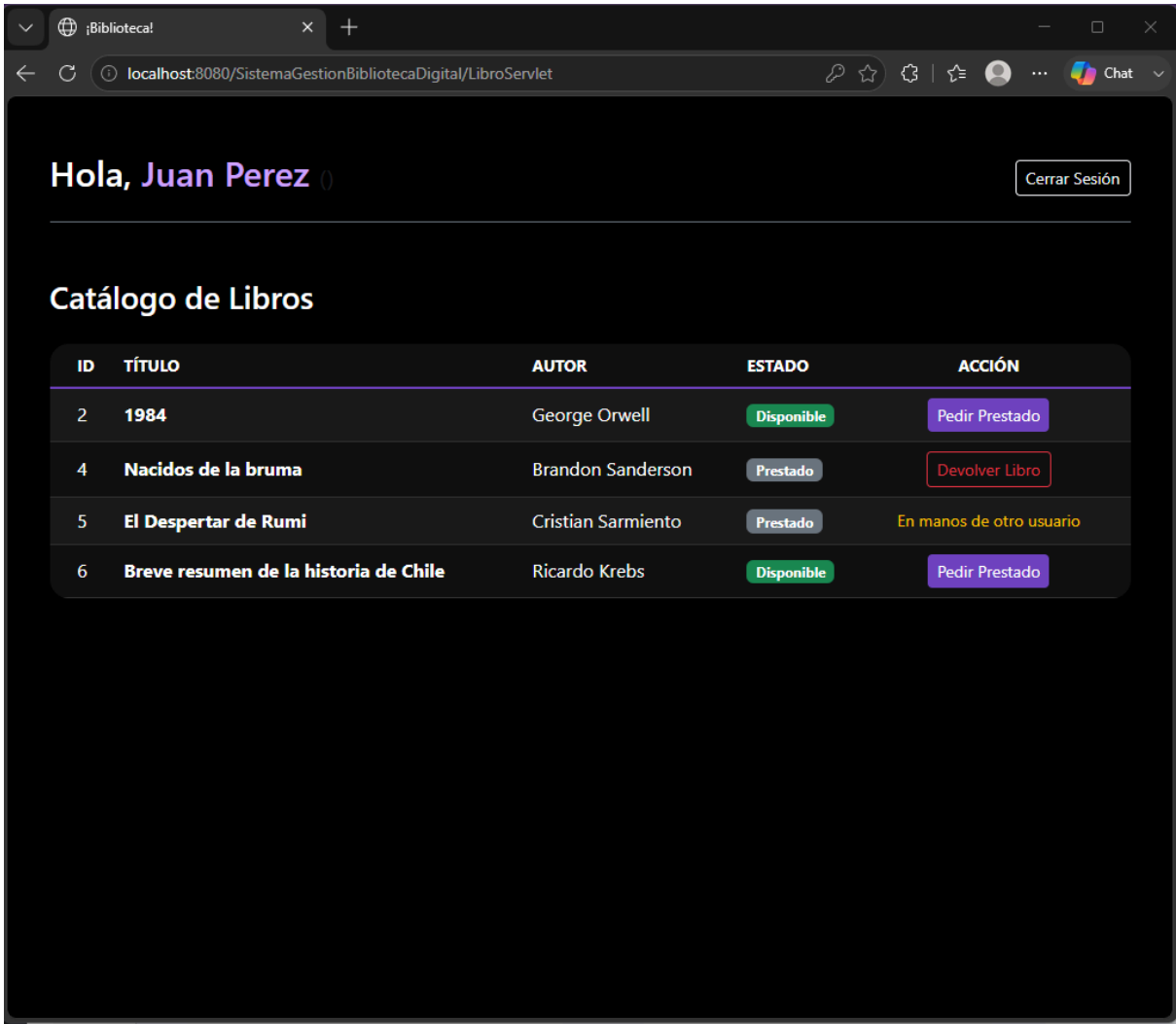
Grosor del dashboard, retornando distinto estado dependiendo del libro, y de si el usuario es administrador o no.

```

119      <div class="d-flex justify-content-center align-items-center gap-2">
120
121          <!-- if libro está disponible, que se pueda pedir -->
122          <% if(l.isDisponible()) { %>
123              <form action="LibroServlet" method="POST" class="m-0">
124                  <input type="hidden" name="accion" value="prestar">
125                  <input type="hidden" name="idLibro" value="<%= l.getId() %>">
126                  <button type="submit" class="btn btn-purple btn-sm">Pedir Prestado</button>
127              </form>
128          <% } else if(l.getIdUsuario() == user.getId()) { %>
129              <form action="LibroServlet" method="POST" class="m-0">
130                  <input type="hidden" name="accion" value="devolver">
131                  <input type="hidden" name="idLibro" value="<%= l.getId() %>">
132                  <button type="submit" class="btn btn-outline-danger btn-sm">Devolver Libro</button>
133              </form>
134          <% } else { %>
135              <% if(user.isEsAdmin()) { %>
136                  <!-- si es admin, puede ver el nombre -->
137                  <span class="text-warning small fw-bold">Prestado a: <%= l.getNombreUsuarioPrestamo() %></span>
138              <% } else { %>
139                  <!-- otro usuario no puede ver quien lo tiene prestado -->
140                  <span class="text-warning small">En manos de otro usuario</span>
141              <% } %>
142          <% } %>
143      </div>

```

Vista de usuario al dashboard. (La vista al administrador fue mostrada anteriormente)



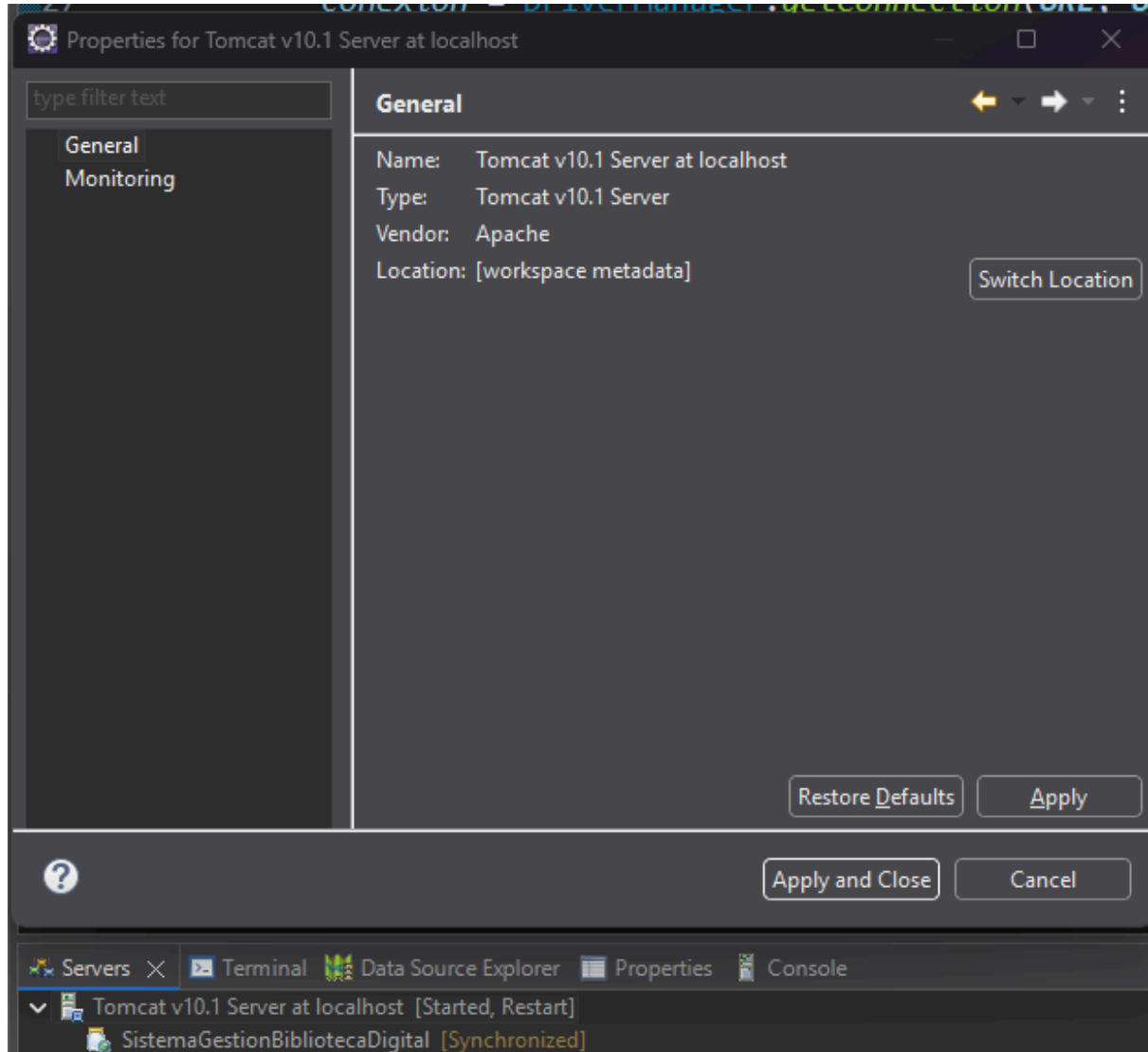
Botón de eliminar para administrador. Se verifica que sea admin. Se hace una doble consulta en dialogo de texto pop-up para no eliminar libros por error.

```
142     <% } %>
143
144     <!-- Boton eliminar si el usuario es admin -->
145     <% if(user.isEsAdmin()) { %>
146         <!-- onsubmit para doble-check -->
147     <form action="LibroServlet" method="POST" class="m-0" onsubmit="return confirm('¿Estás seguro de que deseas eliminar este libro?');">
148         <input type="hidden" name="accion" value="eliminar">
149         <input type="hidden" name="idlibro" value="<%= l.getId() %>">
150         <button type="submit" class="btn btn-danger btn-sm" title="Eliminar Libro">🗑️</button> <!-- icono chistoso intuitivo -->
151     </form>
152     <% } %>
153
154 </div>
155 </td>
156 </tr>
157 <% } %>
158 </tbody>
159 </table>
160 </div>
161 </div>
162 </body>
163 </html>
```

5. Anexo

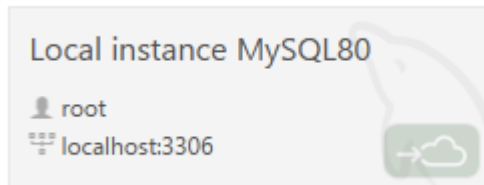
5.1 Tomcat 10.1

instalado usado para la página web dinámica y su uso.



5.2 Base de datos.

Se ocupa una base de datos genérica en MySQLworkbench para la creación, manejo, y levantamiento de la página web dinámica.



usuario: root

contraseña: 1234

Base de datos SQL

```
biblioteca_db x
Limit to 1000 rows
1 • CREATE DATABASE IF NOT EXISTS biblioteca_db;
2 • USE biblioteca_db;
3
4 • CREATE TABLE usuarios (
5     id INT AUTO_INCREMENT PRIMARY KEY,
6     nombre VARCHAR(100) NOT NULL,
7     email VARCHAR(100) UNIQUE NOT NULL,
8     password VARCHAR(100) NOT NULL,
9     es_admin BOOLEAN DEFAULT FALSE
10 );
11
12 • CREATE TABLE libros (
13     id INT AUTO_INCREMENT PRIMARY KEY,
14     titulo VARCHAR(200) NOT NULL,
15     autor VARCHAR(100) NOT NULL,
16     disponible BOOLEAN DEFAULT TRUE,
17     id_usuario INT,
18     FOREIGN KEY (id_usuario) REFERENCES usuarios(id)
19 );
20
21 -- Generar admin por defecto y usuario de prueba
22 • INSERT INTO usuarios (nombre, email, password, es_admin) VALUES
23     ('Administrador/a', 'admin@biblioteca.com', 'admin123', TRUE),
24     ('Juan Perez', 'juan@correo.com', 'user123', FALSE);
25
26 -- Generar libros aleatorios disponibles
27 • INSERT INTO libros (titulo, autor, disponible) VALUES
28     ('El Quijote', 'Miguel de Cervantes', TRUE),
29     ('1984', 'George Orwell', TRUE),
30     ('Dune', 'Frank Herbert', TRUE);
31
32 • SELECT * FROM usuarios;
33 • SELECT * FROM libros;
```

Esta fue la base de datos originalmente creada, con el objetivo de tener un cimiento de donde empezar a trabajar. Libros fueron añadidos posteriormente a través de la página web dinámica, terminando en la base de datos a continuación.

5.2.1 tabla usuarios final.

mismas contraseñas para mantener mi propia sanidad mental.

Result Grid

Filter Rows:

Edit:

Export/Imp

	id	nombre	email	password	es_admin
	1	Administrador	admin@biblioteca.com	admin123	1
	2	Juan Perez	juan@correo.com	user123	0
	3	Nico Krebs	nikrebs@correo.com	user123	0
	4	Patricio Gomez	pago@correo.com	user123	0
	5	123	1231231@corerer	123123	0
▶*	NULL	NULL	NULL	NULL	NULL

5.2.2 tabla libros final

Result Grid

Filter Rows:

Edit:

Export/Import:

	id	titulo	autor	disponible	id_usuario
▶	2	1984	George Orwell	1	NULL
	4	Nacidos de la bruma	Brandon Sanderson	0	2
	5	El Despertar de Rumi	Cristian Sarmiento	0	3
	6	Breve resumen de la historia de Chile	Ricardo Krebs	1	NULL
*	NULL	NULL	NULL	NULL	NULL

Dune de Frank Herbert, fue eliminado por administrador, de ahí nace su ausencia en los pantallazos y en este mismo request final.